



Pattern Recognition
and Applications Lab

La Programmazione ad Oggetti in Python

Docente: Ambra Demontis

Anno Accademico: 2025 - 2026

Corso di Laurea in Ingegneria Elettronica, Informatica e delle Telecomunicazioni



University of Cagliari,
Italy

Department of Electrical and
Electronic Engineering



La Programmazione ad Oggetti in Python

In queste slide vedremo come si implementano le relazioni in Python, in particolare:

- La relazione di composizione
- La relazione di aggregazione
- La relazione di ereditarietà

La relazione di Composizione

Nella lezione precedente abbiamo visto come definire una classe. In particolare abbiamo creato la classe CLibro.

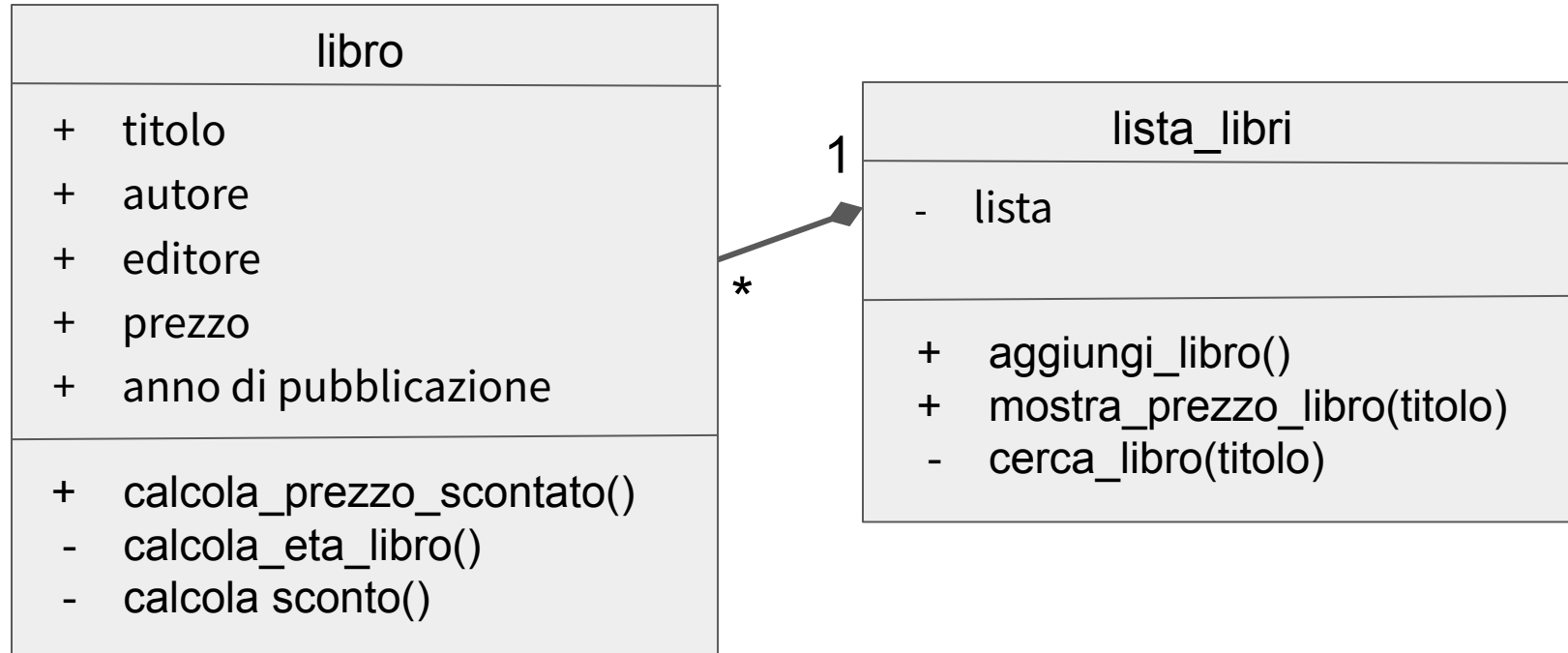
Supponiamo ora di voler memorizzare più libri e quindi di **creare una classe** per gestire una lista dei libri, **composta** da libri appartenenti alla classe CLibro.

Lo scopo della classe che gestisce la lista dei libri è di:

- memorizzare un nuovo libro
- mostrare il prezzo finale (scontato) di un libro

La relazione di Composizione

Il diagramma delle classi del prototipo è il seguente:



Le Strutture Dati di Python e gli Oggetti

Per poter creare questa classe abbiamo bisogno di memorizzare diversi oggetti di tipo libro.

Le strutture dati tupla, lista e dizionario possono memorizzare oggetti.

Possiamo utilizzare ad esempio una lista.

Creiamo la Classe CListaLibri

Il primo metodo che vogliamo implementare è il metodo “aggiungi_libro”.
Come spiegato nella slide precedente è possibile memorizzare gli oggetti in una lista.

Tutti gli attributi, pubblici e privati vanno inizializzati nel metodo `__init__` dell'oggetto.

In questo modo un programmatore che vuole modificare la classe creata guardando il metodo `__init__` è a conoscenza di tutti gli attributi definiti da quella classe.

Le Strutture Dati di Python e gli Oggetti

Che valore gli si deve assegnare quando essi ricevono un valore successivamente?

Nel caso in cui si tratti di:

- Una struttura dati tra: tupla, lista o dizionario, gli si assegna una struttura dati di quel tipo vuota.
- Un altro tipo di dato: gli si assegna il tipo di dato **None** (nessun valore).

L' Operatore Is

Per controllare se una variabile è `None` si utilizza l'operatore `is`.

La sintassi è la seguente:

`<nome_variabile> is None`

Creiamo la Classe CListaLibri

```
class CListaLibri:
```

```
    def __init__(self):
```

```
        self._lista = []
```

Creiamo la Classe CListaLibri

Creiamo il metodo pubblico *aggiungi_libro* che deve permettere di memorizzare un libro nella lista.

Per avere metodi più compatti possiamo far sì che il nuovo libro venga creato utilizzando un apposito metodo.

```
def aggiungi_libro(self):
```

```
    # acquisisce i dati di un nuovo libro e crea un oggetto di tipo CLibro
```

```
    oggetto_libro = self._crea_nuovooggetto_libro()
```

```
    self._lista = self._lista + [oggetto_libro]
```

Creiamo la Classe CListaLibri

La creazione di una nuova istanza di classe libro dovrebbe essere di competenza della classe libro ma con quello che abbiamo visto fino ad ora è impossibile!

Implementiamo il metodo privato `_crea_nuovooggetto_libro` che si occupa di creare un oggetto di tipo libro ed acquisire i valori dei suoi attributi.

```
def _crea_nuovooggetto_libro(self):  
    # acquisisci i dati  
    titolo = input("inserisci il titolo del libro ")  
    autore = input("inserisci l'autore del libro ")  
    editore = input("inserisci l'editore del libro ")  
    prezzo = float(input("inserisci il prezzo del libro "))  
    anno_pubblicazione = int(  
        input("inserisci l'anno di pubblicazione del libro ")
```

```
        # crea un oggetto appartenente alla classe  
        CLibro  
        oggetto_libro = CLibro(titolo, autore,  
                                editore, prezzo,  
                                anno_pubblicazione)  
    return oggetto_libro
```

Creiamo la Classe CListaLibri

Implementiamo il metodo privato `_cerca_libro` che, dato il titolo di un libro, cerca il libro nella lista e ci restituisce l'oggetto corrispondente.

Un modo per implementarlo è il seguente:

```
def _cerca_libro(self, titolo):  
    libro_trovato = False  
    for libro in self._lista:  
        if libro.titolo == titolo:  
            libro_cercato = libro  
            libro_trovato = True
```

```
    if libro_trovato:  
        return libro_cercato  
    else:  
        print("Libro non trovato!")
```

Creiamo la Classe CListaLibri

Questo stesso metodo può essere scritto in modo più conciso sfruttando il fatto che, come nel linguaggio C, quando l'istruzione return viene chiamata, la funzione termina.

```
def _cerca_libro(self, titolo):  
    for libro in self._lista:  
        if libro.titolo == titolo:  
            return libro  
    print("Libro non trovato!")
```

Creiamo la Classe CListaLibri

Infine, dobbiamo implementare il metodo *mostra_prezzo_libro* che, dato il titolo di un libro e l'anno corrente, cerca il libro e poi calcola e mostra il prezzo finale del libro utilizzando il metodo *calcola_prezzo_scontato* della classe Clibro.

```
def mostra_prezzo_libro(self, titolo, anno_corrente):  
    oggetto_libro = self._cerca_libro(titolo)  
    # (quando l'istruzione return non viene chiamata, la funzione restituisce None)  
    if oggetto_libro is not None:  
        print(oggetto_libro.calcola_prezzo_scontato(anno_corrente))
```

Creiamo un'istanza della classe CListaLibri

Proviamo ora ad utilizzare la classe CListalibri aggiungendo due libri e cercando il prezzo scontato di uno dei due libri aggiunti.

```
lista_libri = CListaLibri()  
lista_libri.aggiungi_libro()  
lista_libri.aggiungi_libro()  
lista_libri.mostra_prezzo_libro('libro1', 2021)
```

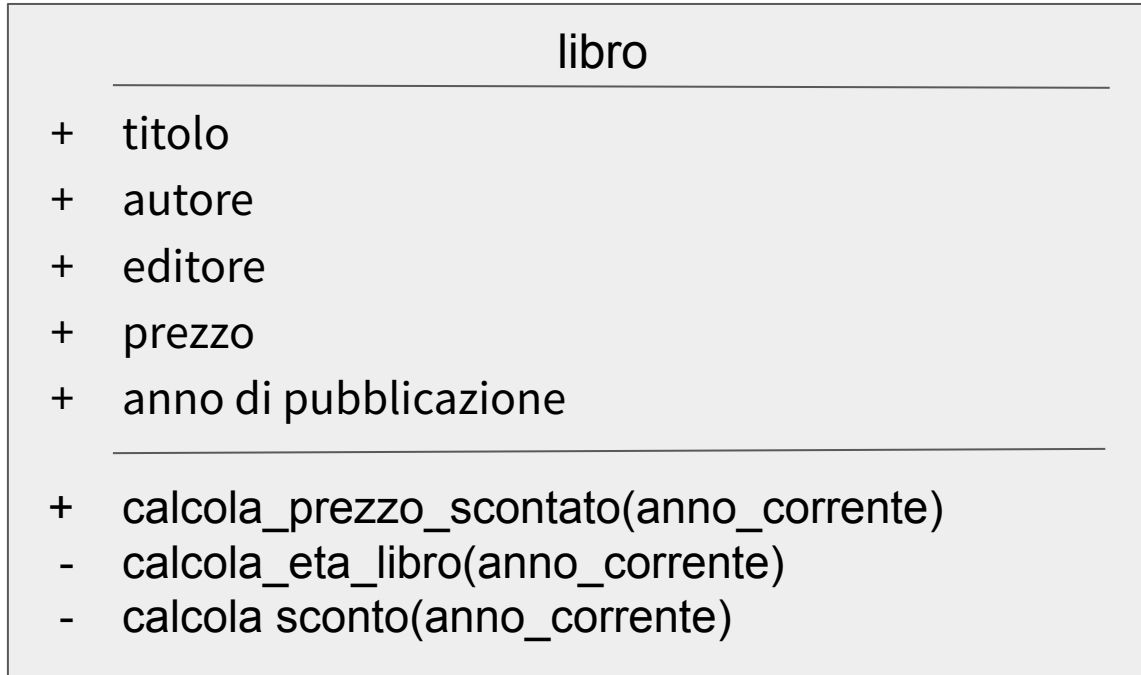
Da Notare sulla Composizione

Come sappiamo si ha una **relazione di composizione** quando **se distruggo l'oggetto composto distruggo anche gli elementi che lo compongono**.

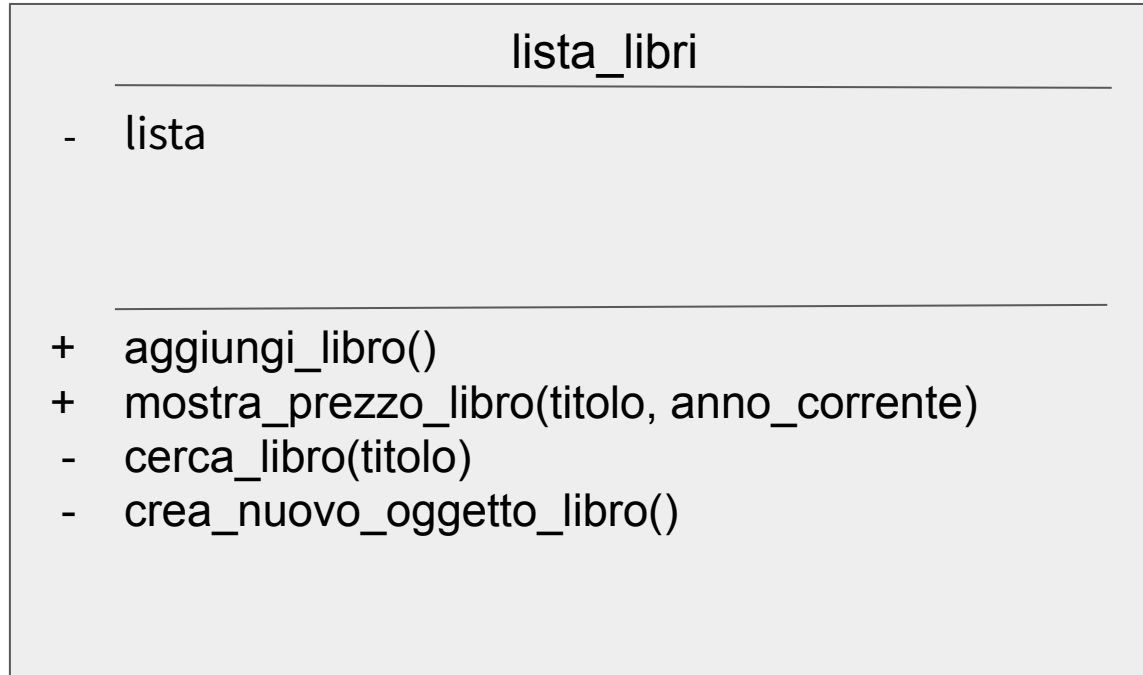
Nell'esempio le istanze dei libri vengono create da metodi della classe CListalibri e vengono memorizzate in un attributo dell'istanza di CListalibri.

Quindi quando distruggo l'istanza di CListalibri non esiste più nessun riferimento alle istanze della classe CLibro e quindi quelle istanze vengono distrutte da Python.

Il Diagramma di Classe della classe CLibro implementata



Il Diagramma di Classe della classe ClistaLibri implementata



Diagrammi di Classe delle Implementazioni e Prototipi

Generalmente:

- I diagrammi delle classi creati come prototipo servono per avere uno schema di massima di ciò che il programmatore dovrà implementare.
- Il programmatore può modificarli per necessità implementative aggiungendo metodi privati per ottenere un codice più compatto, attributi privati o parametri nei metodi.

Diagrammi di Classe delle Implementazioni e Prototipi

A volte però, il programmatore può essere vincolato a rispettare rigorosamente il prototipo o parte di esso.

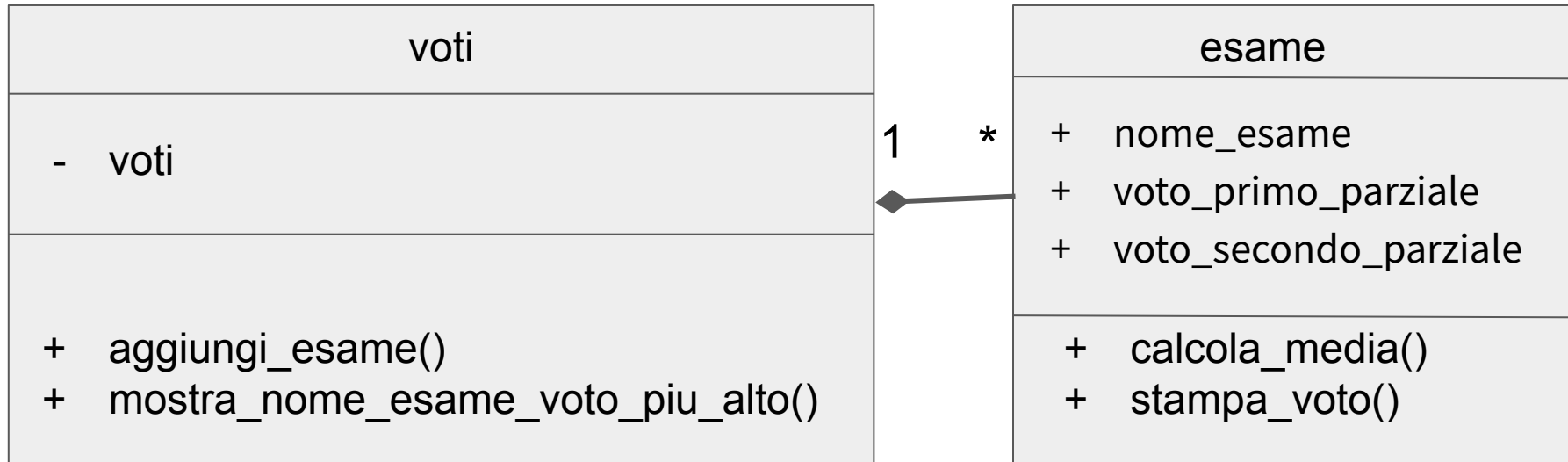
Il caso più comune è quello nel quale il programmatore è obbligato ad utilizzare l'interfaccia specificata per alcuni metodi, in modo che abbiano un' interfaccia comune a quelli di altri oggetti.

Esercizio sulla Composizione in Python

Create la classe *voti*, che deve permettere di:

- Memorizzare diversi oggetti di tipo *CEsame*.
- Stampare il nome dell'esame nel quale è stato ottenuto il voto finale più alto. Nel caso in cui il voto più alto sia lo stesso per più esami, dovrà essere mostrato il nome di tutti gli esami nei quali è stato conseguito quel voto.

Esercizio sulla Composizione in Python



Esercizio sulla Composizione in Python

```
class CEsame:
```

```
    def __init__(self, nome_esame, voto_primo_parziale, voto_secondo_parziale):
```

```
        self.nome_esame = nome_esame
```

```
        self.voto_primo_parziale = voto_primo_parziale
```

```
        self.voto_secondo_parziale = voto_secondo_parziale
```

```
    def calcola_media(self):
```

```
        media = (self.voto_primo_parziale + self.voto_secondo_parziale)/2
```

```
        return media
```

```
    def stampa_voto_finale(self):
```

```
        voto_finale = self.calcola_media()
```

```
        print("Il voto finale per l'esame ", self.nome_esame, " è ", voto_finale)
```

Esercizio sulla Composizione in Python

```
class CVoti:
```

```
    def __init__(self):
```

```
        self._voti = []
```

```
    def aggiungi_esame(self):
```

```
        # acquisisce i dati di un nuovo esame e crea un oggetto di tipo CEsame
```

```
        oggetto_esame = self._crea_nuovo_oggetto_esame()
```

```
        self._voti = self._voti + [oggetto_esame]
```

```
... continua
```


Esercizio sulla Composizione in Python

```
def _crea_nuovooggetto_esame(self):
```

```
    # acquisisci i dati
```

```
    nome_esame = input("inserisci il nome dell'esame ")
```

```
    voto_primo_parziale = int(input("inserisci il voto conseguito nel primo parziale "))
```

```
    voto_secondo_parziale = int(input("inserisci il voto conseguito nel primo parziale "))
```

```
    # crea un oggetto appartenente alla classe CEsame
```

```
    oggetto_esame = CEsame(nome_esame, voto_primo_parziale, voto_secondo_parziale)
```

```
    return oggetto_esame
```

... continua

Esercizio sulla Composizione in Python

Versione 1:

```
def mostra_nome_esame_voto_piu_alto(self):
```

```
    # cerca il voto più alto
```

```
    max_voto = 0
```

```
    for esame in self._voti:
```

```
        voto_finale = esame.calcola_media()
```

```
        if voto_finale > max_voto:
```

```
            max_voto = voto_finale
```

```
    # stampa i nomi degli esami nei quali è  
    # stato conseguito il voto più alto
```

```
    for esame in self._voti:
```

```
        voto_finale = esame.calcola_media()
```

```
        if voto_finale == max_voto:
```

```
            print(esame.nome_esame)
```

Esercizio sulla Composizione in Python

Versione 2:

```
def mostra_nome_esame_voto_piu_alto(self):
```

```
    # cerca il voto più alto
```

```
    nomi_esami_voto_piu_alto = []
```

```
    max_voto = 0
```

```
    for esame in self._voti:
```

```
        voto_finale = esame.calcola_media()
```

```
        if voto_finale > max_voto:
```

```
            max_voto = voto_finale
```

```
            nomi_esami_voto_piu_alto = [esame.nome_esame]
```

```
        else:
```

```
            if voto_finale == max_voto:
```

```
                nomi_esami_voto_piu_alto += [esame.nome_esame]
```

```
    for nome_esame_voto_piu_alto in
```

```
        nomi_esami_voto_piu_alto:
```

```
        print(nome_esame_voto_piu_alto)
```

Esercizio sulla Composizione in Python

Proviamo ad utilizzare la classe creata:

```
lista_voti = CVoti()  
lista_voti.aggiungi_esame()  
lista_voti.aggiungi_esame()  
lista_voti.aggiungi_esame()  
lista_voti.mostra_nome_esame_voto_piu_alto()
```

La Relazione di Aggregazione

Si ha una relazione di aggregazione quando un oggetto mette insieme altri oggetti che hanno una vita propria: oggetti che devono continuare ad esistere se l'oggetto che li aggrega viene cancellato.

La Relazione di Aggregazione

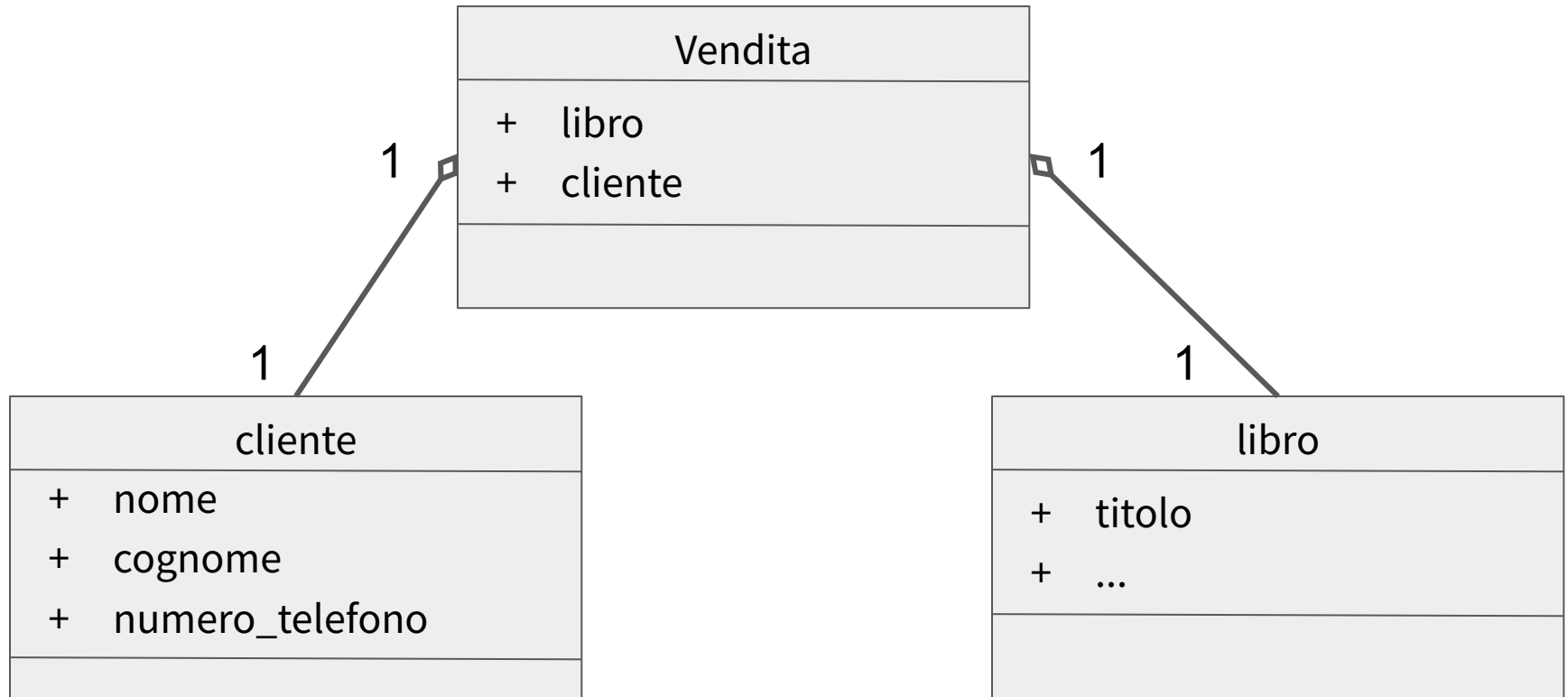
Supponete di venire assunti dal gestore di una libreria per espandere il programma che utilizza per gestire la libreria. Quel programma, memorizza:

- I dati dei libri dalla libreria nella classe CListaLibri vista precedentemente.
- I dati dei clienti in una classe CListaClienti composta da oggetti di classe CCliente, che hanno gli attributi: nome, cognome, numero_di_telefono.

Il gestore della libreria vi chiede di creare una classe CVendita che permetta di memorizzare, per vendita di un singolo libro, quale libro è stato venduto e a quale cliente è stato venduto.

Questa classe deve poter essere rimossa senza che vengano eliminati i libri venduti o i dati dei clienti e non deve duplicare dati.

La Relazione di Aggregazione



La Relazione di Aggregazione

La classe CVendita memorizzerà un riferimento ad un oggetto di tipo cliente e ad un oggetto libro esistenti.

```
class CVendita:
```

```
    def __init__(self, cliente, libro):  
        self.cliente = cliente  
        self.libro = libro
```


La Relazione di Aggregazione

Un esempio di codice che crea un oggetto di classe CVendita è il seguente:

```
titolo = input("inserisci il titolo del libro ")
autore = input("inserisci l'autore del libro ")
editore = input("inserisci l'editore del libro ")
prezzo = float(input("inserisci il prezzo del libro "))
anno_pubblicazione = int(input("inserisci l'anno di pubblicazione del libro "))
oggetto_libro = CLibro(titolo, autore, editore, prezzo, anno_pubblicazione)

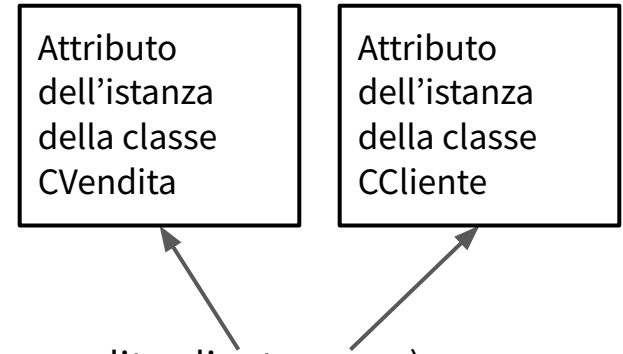
oggetto_cliente = CCliente("Anna", "Bianchi", "070889977")

oggetto_vendita = CVendita(oggetto_libro, oggetto_cliente)
```

La Relazione di Aggregazione

Come spiegato, nell'oggetto CVendita vengono memorizzati dei riferimenti ad un oggetto libro e ad un oggetto cliente esistenti. Possiamo quindi accedere agli attributi di quegli oggetti.

```
oggetto_vendita = CVendita(oggetto_libro, oggetto_cliente)
```



```
print("Il nome del cliente che ha comprato il libro è ", oggetto_vendita.cliente.nome)
```

Stamperà: Il nome del cliente che ha comprato il libro è Anna

La Relazione di Aggregazione

Se eliminassi l'istanza di classe CVendita esisterebbe comunque almeno un riferimento agli oggetti di classe libro e cliente e quindi non verrebbero eliminati.

La Relazione di Aggregazione

Nb: Poichè la classe CVendita non crea al suo interno le istanze, queste potrebbero essere cancellate da fuori rispetto a quella classe.

Se essi cessassero di esistere questa classe non funzionerebbe più correttamente.

I.e. se venissero cancellate utilizzando la funzione `del` (<istanza>)

La Relazione di Ereditarietà

Come abbiamo visto delle classi possono essere legate da una relazione di ereditarietà. In questo caso **le classi figlie ereditano attributi e metodi definiti nella classe padre.**

La Relazione di Ereditarietà

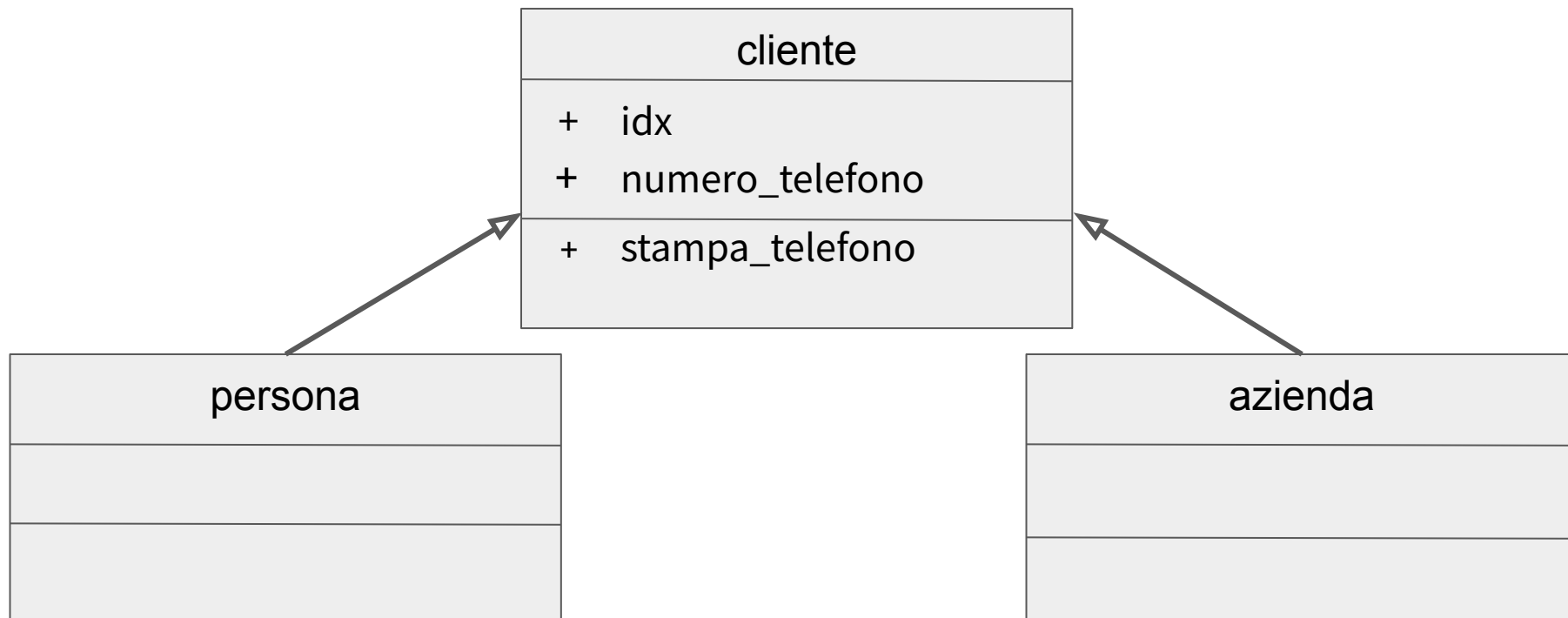
Supponiamo di voler creare tre classi, le classi: *persona*, *azienda* e *cliente*.

Vogliamo che tutte abbiano due attributi: *idx* e *numero_telefono*.

Vogliamo inoltre che entrambe abbiano un metodo chiamato *stampa_telefono* che deve stampare:

Il numero di telefono di <idx> è <numero_telefono>

La Relazione di Ereditarietà



La Relazione di Ereditarietà

Per far sì che una classe erediti da un'altra dobbiamo usare la seguente sintassi:

```
class <nome_classe_figlia> (<nome_classe_padre>):  
    ...
```


Creazione della Classe Padre

Il codice corrispondente al diagramma delle classi visto prima sarà il seguente:

```
class CCliente:
    def __init__(self, idx, numero_telefono):
        self.idx = idx
        self.numero_telefono = numero_telefono

    def stampa_telefono(self):
        print("Il numero di telefono di ", self.idx, " è: ", self.numero_telefono)
```

Creazione delle Classi Figlie

In questo esempio le classi figlie **ereditano tutto dalla classe padre**.

Dovremo quindi semplicemente crearle indicando che sono figlie della classe CCliente.

```
class CAzienda(CCliente):  
    pass
```

```
class CPersona(CCliente):  
    pass
```

Poichè le queste classi non devono fare nulla utilizziamo l'istruzione `pass`. Questa istruzione non fa nulla, serve a riempire un blocco di codice previsto dalla sintassi.

(Se provassimo a rimuovere l'istruzione `pass` riscontreremmo un errore di sintassi.)

Inizializzazione di un Oggetto di Classe CAzienda

Creiamo un oggetto di tipo CAzienda e utilizziamo il metodo *stampa_telefono* che viene ereditato dalla classe padre CCliente.

Esempio:

```
oggetto_azienza = CAzienda("123", "070998899")  
oggetto_azienza.stampa_telefono()
```

Questo codice stamperà a schermo:

Il numero di telefono di 123 è: 070998899

Questo mostra che la classe CAzienda effettivamente ha ereditato il metodo *stampa_telefono* dal padre.

Overriding di un Metodo Definito dalla Classe Padre

Le classi figlie ereditano tutti i metodi dalla classe padre. Tuttavia è **possibile modificare il codice eseguito da un metodo ereditato** dalla classe padre **sovrascrivendo la definizione del metodo** (creando un metodo che abbia lo stesso nome ma un comportamento differente).

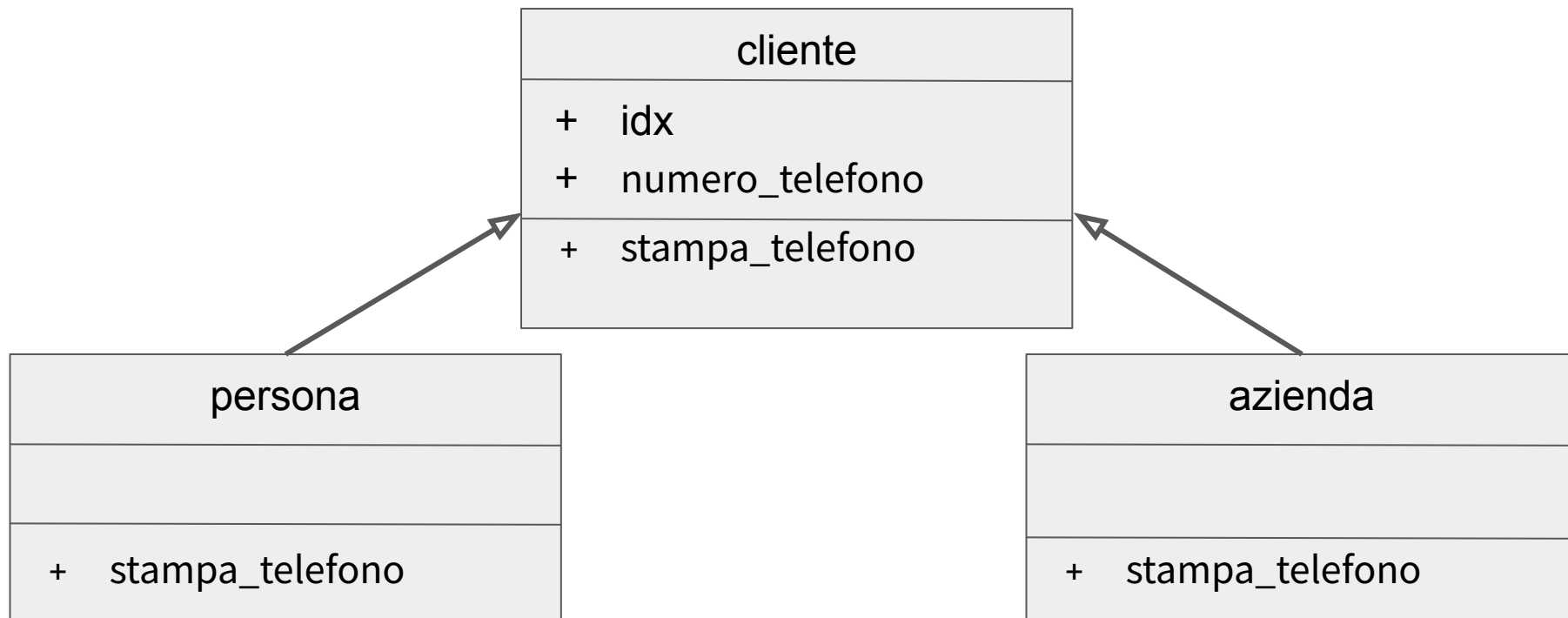
Overriding di un Metodo Definito dalla Classe Padre

Supponiamo ora di voler far sì che il metodo `stampa_telefono` stampi:

- Per la classe `persona`:
Il numero di telefono della persona con `<idx>` è `<numero_telefono>`
- Per la classe `azienda`:
Il numero di telefono dell'azienda con `<idx>` è `<numero_telefono>`

Per far questo supponiamo di voler sovrascrivere nei figli il metodo `stampa_telefono` definita dalla classe padre.

La Relazione di Ereditarietà



Overriding di un Metodo Definito dalla Classe Padre

Il codice della classe CCliente rimarrà identico a quello visto in precedenza, mentre **cambieranno le definizioni delle classi figlie in quanto devono sovrascrivere il metodo *stampa_telefono* in modo da cambiarne il comportamento.**

```
class CAzienda(CCliente):
```

```
    def stampa_telefono(self):  
        print("Il numero di telefono dell'azienda con ", self.idx, " è:",  
              self.numero_telefono)
```

Overriding di un Metodo Definito dalla Classe Padre

```
class CPersona(CCliente):
```

```
    def stampa_telefono(self):
```

```
        print("Il numero di telefono della persona con ", self.idx, " è: ",  
              self.numero_telefono)
```


Overriding di un Metodo Definito dalla Classe Padre

Proviamo ora a creare una oggetto di classe CAzienda:

```
oggetto_azienza = CAzienda("123", "070998899")  
oggetto_azienza.stampa_telefono()
```

Stamperà a schermo:

Il numero di telefono dell'azienda con 123 è: 070998899

Come vediamo quello che viene richiamato è il metodo *stampa_telefono* definito nella classe CAzienda e non più nella classe CCliente.

La Relazione di Ereditarietà

Gli attributi necessari sono differenti a seconda del tipo di cliente quindi dovremo avere due classi (una per le persone e una per le aziende).

Notiamo anche che hanno degli attributi in comune:

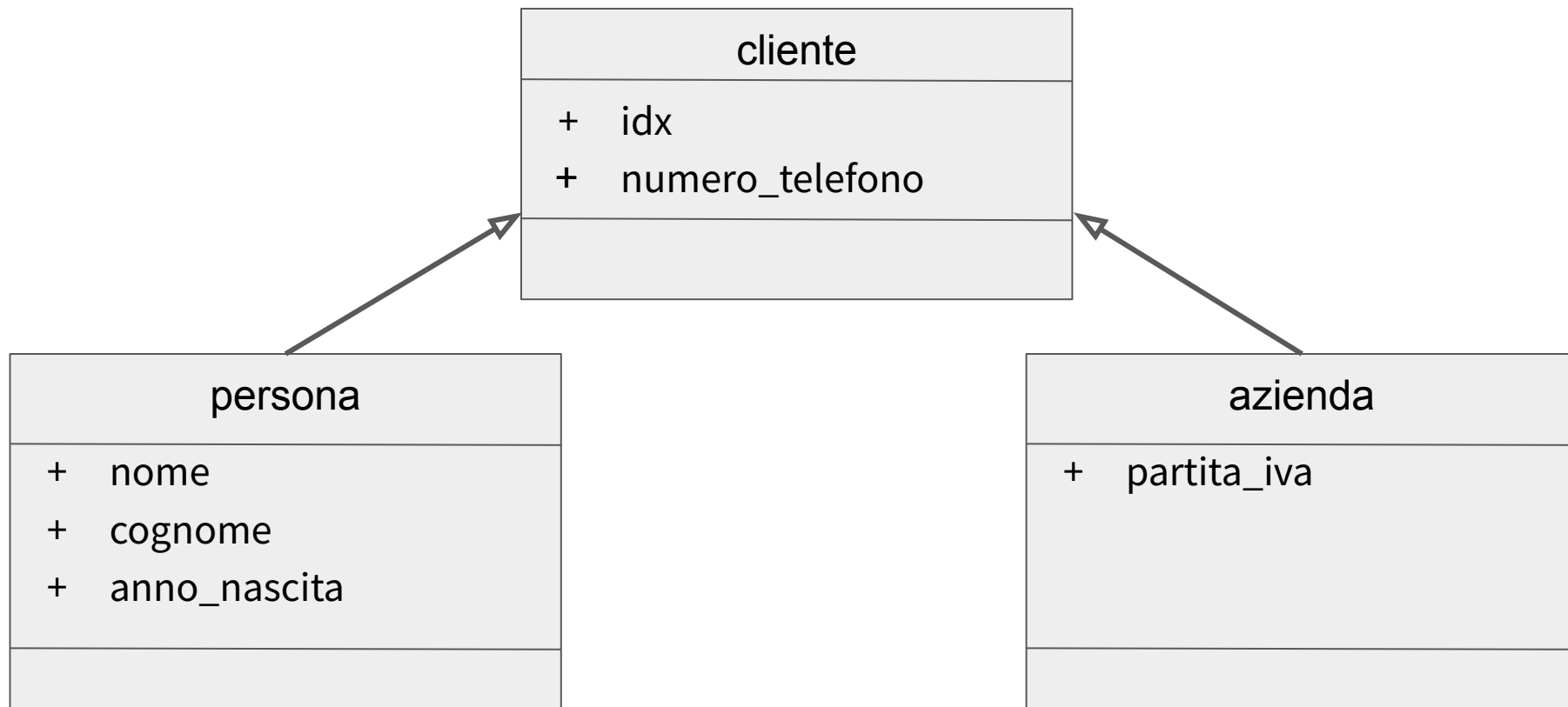
Per le persone: idx, nome, cognome, anno di nascita, numero di telefono.

Per le aziende: idx, partita iva, numero di telefono.

Dove idx è un identificativo univoco assegnato dall'azienda ai suoi clienti.

Quindi ha senso creare una terza classe “cliente” che sarà la classe padre.

La Relazione di Ereditarietà



La Classe Padre CCliente

```
class CCliente:  
    def __init__(self, idx, numero_telefono):  
        self.idx = idx  
        self.numero_telefono = numero_telefono
```

cliente	
+	idx
+	numero_telefono

La Classe Figlia CAzienda

La classe figlia CAzienda deve occuparsi di memorizzare solo l'attributo partita_iva in quanto gli altri vengono memorizzati dalla classe padre. Ci si potrebbe quindi aspettare di poter definire la classe CAzienda così:

```
class CAzienda(CCliente):  
  
    def __init__(self, partita_iva):  
        self.partita_iva = partita_iva
```

Questo però non è il modo corretto di definirla...

La Classe Figlia CAzienda

```
class CAzienda(CCliente):  
  
    def __init__(self, partita_iva):  
        self.partita_iva = partita_iva
```

La classe CCliente acquisisce questi attributi utilizzando il metodo `__init__`.

Init è un metodo. Questo codice sovrascrive il metodo `__init__` definito nella classe padre. Verrebbe quindi memorizzato solo l'attributo *partita_iva* e non verrebbero memorizzati *idx* e *numero_telefono*.

Richiamare un Metodo della Classe Padre

Poichè abbiamo sovrascritto il metodo `__init__`, se vogliamo sfruttare il fatto che il metodo `__init__` della classe padre (CCliente) definisce alcuni attributi dobbiamo richiamarlo esplicitamente.

Per richiamare esplicitamente il metodo `__init__` della classe padre si utilizza:
`super().__init__(<argomento1>..`

Genericamente, per richiamare esplicitamente un metodo della classe padre si può usare la sintassi:

`super().<nome_metodo>(<argomento1>..`

Richiamare un Metodo della Classe Padre

Aggiungiamo la chiamata al metodo `__init__` della classe padre.

```
class CAzienda(CCliente):  
  
    def __init__(self, partita_iva):  
        self.partita_iva = partita_iva  
        super().__init__(idx, numero_telefono)
```


Richiamare un Metodo della Classe Padre

NB: Per poter passare all'init della classe padre i valori degli attributi idx e numero_telefono dobbiamo far si che la classe riceva anche questi attributi al momento della creazione dell'oggetto. Dobbiamo quindi aggiungere i relativi parametri nel metodo `__init__` della classe figlia.

```
class CAzienda(CCliente):
```

```
    def __init__(self, partita_iva, idx, numero_telefono):  
        self.partita_iva = partita_iva  
        super().__init__(idx, numero_telefono)
```

Richiamare un Metodo della Classe Padre

Quando creiamo un'istanza della classe CAzienda dovremmo quindi passare come argomento sia i valori degli attributi settati dalla classe CAzienda che quelli settati dalla classe padre CCliente.

```
oggetto_azienza = CAzienda("12345678901", "123", "070998899")  
print("La partita iva e' ", oggetto_azienza.partita_iva)  
print("Il numero di telefono e' ", oggetto_azienza.numero_telefono)
```

Stamperà:

La partita iva e' 12345678901

Il numero di telefono e' 070998899

La Classe CCliente

La classe CPersona sarà molto simile alla classe CAzienda ma definirà attributi differenti:

```
class CPersona(CCliente):
```

```
    def __init__(self, nome, cognome, anno_nascita, idx, numero_telefono):
```

```
        self.nome = nome
```

```
        self.cognome = cognome
```

```
        self.anno_nascita = anno_nascita
```

```
        super().__init__(idx, numero_telefono)
```

Richiamare un Metodo della Classe Padre

Leggendo vecchi codici Python potreste trovare la sintassi:

```
super(<nome_classe>, self).__init__(<argomento1>..<argomenton>)
```

Questa sintassi è equivalente a:

```
super().__init__(<argomento1>..<argomenton>)
```

Esercizio sull'Utilizzo delle Relazioni

Creare un programma che permetta di memorizzare temporaneamente i dati anagrafici di studenti e insegnanti di una scuola superiore.

Gli attributi da memorizzare sono:

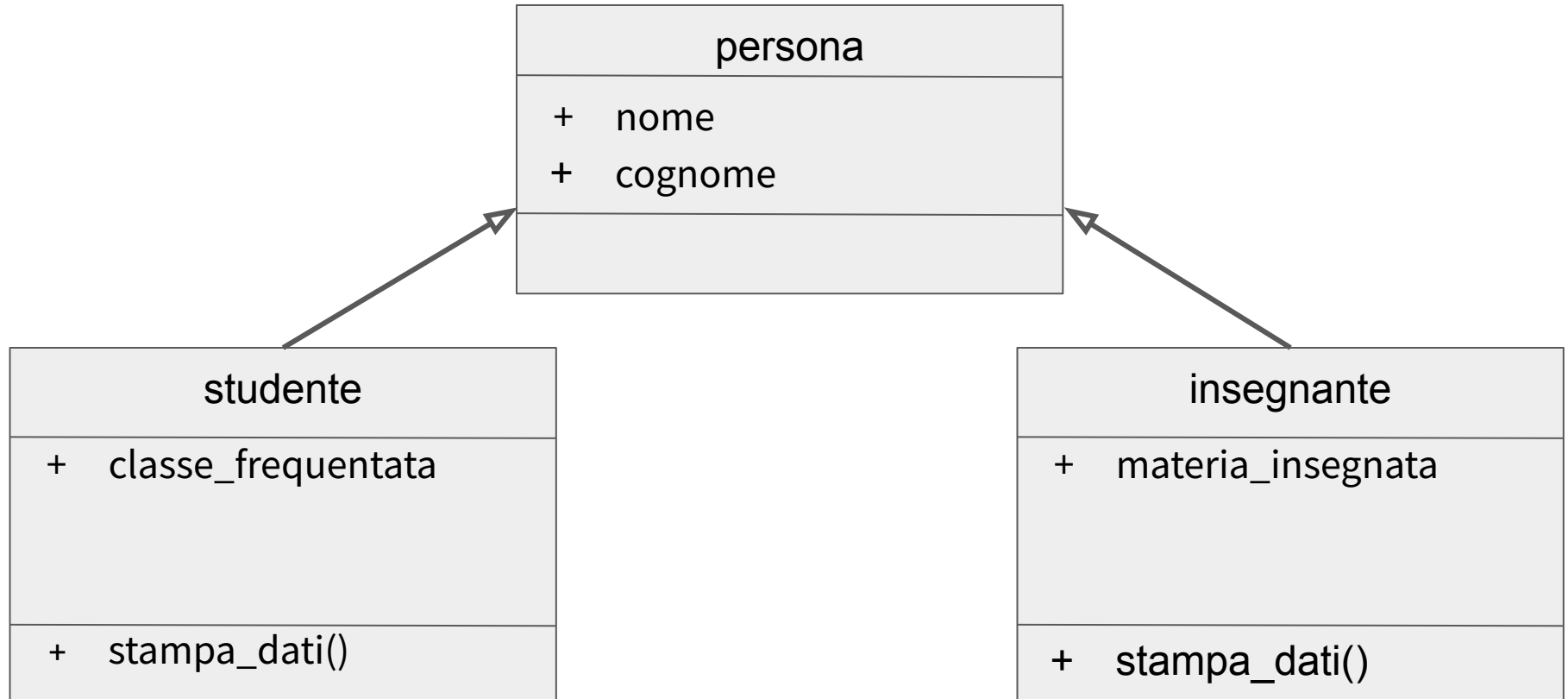
Per gli studenti: matricola, nome, cognome, anno di nascita, classe_frequentata.

Per i docenti: nome, cognome, materia_insegnata.

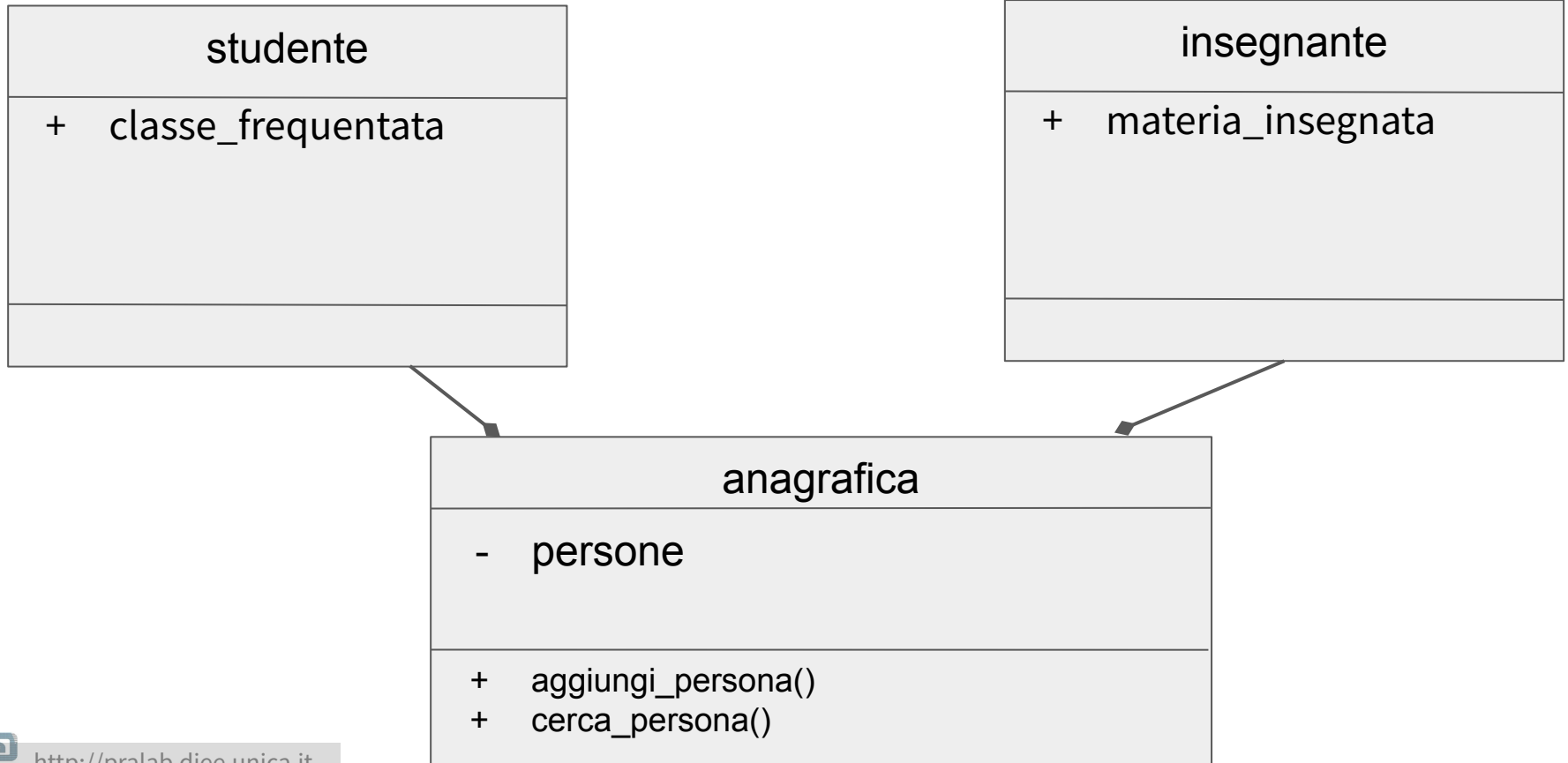
Il programma deve permettere, acquisiti in input il nome e il cognome di cercare i dati di uno studente o del docente e di stamparli.

Creare prima il diagramma delle classi.

Esercizio sull'Utilizzo delle Relazioni



Esercizio sull'Utilizzo delle Relazioni



Esercizio sull'Utilizzo delle Relazioni

Per semplificare il codice con le scelte dell'utente potete usare l'if-elif-else.

Sintassi:

If <condizione>:

...

elif <condizione>:

...

else:

...

Esercizio sull'Utilizzo delle Relazioni

```
class CPersona:
    def __init__(self, nome, cognome):
        self.nome = nome
        self.cognome = cognome

class CStudente(CPersona):
    def __init__(self, nome, cognome, classe_frequentata):
        self.classe_frequentata = classe_frequentata
        super().__init__(nome, cognome)

    def stampa_dati(self):
        print("classe frequentata ", self.classe_frequentata)
```

Esercizio sull'Utilizzo delle Relazioni

```
class CInsegnante(CPersona):  
    def __init__(self, nome, cognome, materia_insegnata):  
        self.materia_insegnata = materia_insegnata  
        super().__init__(nome, cognome)  
  
    def stampa_dati(self):  
        print("materia insegnata ", self.materia_insegnata)
```

Esercizio sull'Utilizzo delle Relazioni

```
class CAnagrafica():  
    def __init__(self):  
        self._persone = []  
  
    def _crea_insegnante(self):  
        nome = input ("inserisci il nome ")  
        cognome = input ("inserisci il cognome ")  
        materia_insegnata = input ("inserisci il nome della materia ")  
        return CInsegnante(nome, cognome, materia_insegnata)  
  
    def _crea_studente(self):  
        nome = input ("inserisci il nome ")  
        cognome = input ("inserisci il cognome ")  
        classe_frequentata = input ("inserisci la classe frequentata ")  
        return CStudente(nome, cognome, classe_frequentata)
```

Esercizio sull'Utilizzo delle Relazioni

```
def aggiungi_persona(self):
    tipo = input("vuoi inserire un insegnante o uno studente? digita 'insegnante' o  
'studente' ")
    if tipo == 'insegnante':
        persona = self._crea_insegnante()
    else:
        if tipo == 'studente':
            persona = self._crea_studente()
        else:
            print("tipo errato")
            return

    self._persone = self._persone + [persona]
```

Esercizio sull'Utilizzo delle Relazioni

```
def cerca_persona(self):  
    nome = input("inserisci il nome della persona che vuoi cercare ")  
    cognome = input("inserisci il cognome della persona che vuoi cercare ")  
  
    i = 0  
    while i < len(self._persone):  
        persona = self._persone[i]  
        if persona.nome == nome and persona.cognome == cognome:  
            self._persone[i].stampa_dati()  
        i = i + 1
```

Esercizio sull'Utilizzo delle Relazioni

```
anagrafica = CAnagrafica()
scelta = 1

while scelta != 3:
    scelta = int(input("Cosa vuoi fare? Inserisci 1 per aggiungere una persona, inserire 2
per cercare una persona, 3 per terminare il programma "))
    if scelta == 1:
        anagrafica.aggiungi_persona()
    elif scelta == 2:
        anagrafica.cerca_persona()
    elif scelta == 3:
        pass
    else:
        print("la scelta inserita è errata")
```

Esercizio sull'Utilizzo delle Relazioni

Se avessimo il codice fiscale (o un'altro attributo univoco) ci converrebbe utilizzare un dizionario, in questo modo non dovremmo implementare la ricerca che verrebbe eseguita in modo più efficiente.

Esercizio sull' Utilizzo delle Relazioni

La classe padre CPersona:

```
class CPersona:
    def __init__(self, codice_fiscale):
        self.codice_fiscale = codice_fiscale
```


Esercizio sull' Utilizzo delle Relazioni

La classe figlia CStudente:

```
class CStudente(CPersona):  
    def __init__(self, codice_fiscale, classe_frequentata):  
        self.classe_frequentata = classe_frequentata  
        super().__init__(codice_fiscale)  
  
    def stampa_dati(self):  
        print("classe frequentata ", self.classe_frequentata)
```

Esercizio sull' Utilizzo delle Relazioni

La classe figlia CInsegnante:

```
class CInsegnante(CPersona):  
    def __init__(self, codice_fiscale, materia_insegnata):  
        self.materia_insegnata = materia_insegnata  
        super().__init__(codice_fiscale)  
  
    def stampa_dati(self):  
        print("materia insegnata ", self.materia_insegnata)
```

Esercizio sull' Utilizzo delle Relazioni

```
class CAnagrafica():  
  
    def __init__(self):  
        self._persone = {}  
  
    def _crea_insegnante(self):  
        codice_fiscale = input("inserisci il codice_fiscale ")  
        materia_insegnata = input("inserisci il nome della materia ")  
        return CInsegnante(codice_fiscale, materia_insegnata)  
  
    def _crea_studente(self):  
        codice_fiscale = input("inserisci il codice_fiscale ")  
        classe_frequentata = input("inserisci la classe frequentata ")  
        return CStudente(codice_fiscale, classe_frequentata)
```

Esercizio sull' Utilizzo delle Relazioni

```
def aggiungi_persona(self):  
  
    tipo = input("vuoi inserire un insegnante o uno studente? digita  
'insegnante' o 'studente' ")  
  
    if tipo == 'insegnante':  
        persona = self._crea_insegnante()  
    else:  
        if tipo == 'studente':  
            persona = self._crea_studente()  
        else:  
            print("tipo errato")  
            return  
  
    self._persone[persona.codice_fiscale] = persona
```

Esercizio sull' Utilizzo delle Relazioni

```
def cerca_persona(self):  
    codice_fiscale = input("inserisci il codice fiscale della  
persona che vuoi cercare ")  
    self._persone[codice_fiscale].stampa_dati()
```

Esercizio sull' Utilizzo delle Relazioni

```
anagrafica = CAnagrafica()
scelta = 1
while scelta != 3:
    scelta = int(input("Cosa vuoi fare? Inserisci 1 per aggiungere una persona, inserire 2
per cercare una persona, 3 per terminare il programma "))
    if scelta == 1:
        anagrafica.aggiungi_persona()
    elif scelta == 2:
        anagrafica.cerca_persona()
    elif scelta == 3:
        pass
    else:
        print("la scelta inserita è errata")
```

Overloading

Alcune volte può essere comodo avere un metodo che si comporta in modo diverso a seconda del tipo di parametri che riceve. La funzionalità che permette di avere metodi con un'implementazione differente a seconda del tipo di dati in input si chiama **overloading**.

E' presente in Java.

Non è presente di per se in Python ma c'è un modo per ottenere qualcosa di simile.

Overloading (Java)

```
class CreditCard {  
    String numero_carta;  
    String nome_proprietario;  
  
    CreditCard(String numero_carta, String nome_proprietario) {  
        this.numero_carta = numero_carta;  
        this.nome_proprietario = nome_proprietario;  
    }  
}  
  
class PayPalAccount {  
    String email;  
  
    PayPalAccount(String email) {  
        this.email = email;  
    }  
}
```


Overloading (Java)

```
class ServiziDiPagamento {  
  
    // Process payment with credit card  
    public void processa_pagamento (CreditCard card, double cifra) {  
        System.out.println("Sto addebitando sulla carta numero " + card.numero_carta +  
            " appartenente a " + card.nome_proprietario +  
            " per $" + cifra);  
    }  
  
    // Process payment with PayPal  
    public void processa_pagamento (PayPalAccount account, double cifra) {  
        System.out.println("Sto addebitando sul PayPal account " + account.email +  
            " per $" + cifra);  
    }  
}
```

Overloading (Java)

```
public class Main {  
    public static void main(String[] args) {  
  
        ServiziodiPagamento paymentService = new ServiziodiPagamento();  
  
        CreditCard card = new CreditCard("1234-5678-9999", "Alice");  
        PayPalAccount paypal = new PayPalAccount("alice@example.com");  
  
        paymentService.processa_pagamento(card, 50.0);  
        paymentService.processa_pagamento(paypal, 25.0);  
    }  
}
```

Verrà stampato:

Sto addebitando sulla carta numero Alice appartenente a Alice per \$50.0

Sto addebitando sul PayPal account alice@example.com per \$25.0

Overloading forzato (Python)

```
from functools import singledispatchmethod

class CCreditCard:
    def __init__(self, card_number, holder_name):
        self.card_number = card_number
        self.holder_name = holder_name

class CPayPalAccount:
    def __init__(self, email):
        self.email = email
```

Forzato perchè è necessario utilizzare i decorator e si basa solo sul primo parametro passato dall'utente.

Overloading forzato (Python)

```
class CServiziDiPagamento:
```

```
    @singledispatchmethod
```

```
    def processa_pagamento(self, metodo_pagamento, cifra):  
        print("Metodo di pagamento non supportato")
```

```
    @processa_pagamento.register
```

```
    def _(self, metodo_pagamento: CCreditCard, cifra):  
        print("Sto addebitando sulla carta numero ", metodo_pagamento.card_number,  
              "appartenete a ", metodo_pagamento.holder_name, " for ", cifra)
```

```
    @processa_pagamento.register
```

```
    def _(self, metodo_pagamento: CPayPalAccount, cifra):  
        print("Sto addebitando sul PayPal account", metodo_pagamento.email, " per ",  
cifra)
```

Processa pagamento dovrebbe essere un metodo di classe.. li vedremo più avanti

“@<nome>” sono “decoratori”. I decoratori sono uno strumento per alterare il comportamento di tutte le funzioni che vengono “decorate”.

Overloading forzato (Python)

```
card = CCreditCard( "1234-5678-9999", "Alice")  
paypal = CPayPalAccount( "alice@example.com")
```

```
servizio_pagamento = CServiziDiPagamento()  
servizio_pagamento.processa_pagamento(card, 50)  
servizio_pagamento.processa_pagamento(paypal, 25)
```

Esercizio sull' Overloading forzato

Crea una classe chiamata formattatore che data una delle seguenti strutture dati di python: lista e dizionario ne stampa i valori in modo user-friendly.

Data una lista, es [1,2,3] dovrà stampare:

Gli elementi della lista sono:

1, 2, 3

Dato un dizionario, es: {"a":1, "b":2} dovrà stampare:

Gli elementi del dizionario sono:

a=1, b=2

Può esservi d'aiuto la funzione enumerate.

Esercizio sull' Overloading forzato

```
dati = ["a", "b", "c"]
```

```
for idx, val in enumerate(dati):  
    print(idx, " ", val)
```

Stamperà:

0 a

1 b

2 c

Esercizio sull' Overloading forzato

```
from functools import singledispatchmethod

class CFormattatore:

    @singledispatchmethod
    def formatta(self, dato):
        print("tipo non supportato")
```


Esercizio sull' Overloading forzato

```
@formatta.register
def _(self, dato: list):
    print("Gli elementi della lista sono: ")
    stringa = ""
    for i,d in enumerate(dato):
        if i == 0:
            stringa = stringa + str(i)
        else:
            stringa = stringa + "," + str(i)
    print(stringa)
```

Esercizio sull' Overloading forzato

```
@formatta.register
def _(self, dato: dict):
    print("Gli elementi del dizionario sono: ")
    stringa = ""
    for i, chiave in enumerate(dato):
        if i != 0:
            stringa = stringa + ","
        stringa = stringa + str(chiave) + " = " + str(dato[chave])
    print(stringa)
```

```
formattatore = CFormattatore()
formattatore.formatta([1,2,3])
formattatore.formatta({"a":1, "b":2})
```

Stamperà:

Gli elementi della lista sono:

0,1,2

Gli elementi del dizionario sono:

a = 1,b = 2